

Accenture Most Asked Python DSA Questions

(Covers Arrays, Strings, Linked List, Stack, Queue, Trees, BST, DP & More)

Created by: Abhishek Rathor

Instagram: `SYNTAX ERROR (code.abhii07)`

Key Features:

- **75+ Handpicked Interview Questions** from real coding rounds & assessments.
- **Beginner to Advanced Topics:**
 - Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Dynamic Programming → Advanced Backtracking & Trie.
- **C++ Implementations** with well-structured **Time & Space Complexities**.
- **Concept Explanations** and **Optimal Approaches** for quick revision.
- **Optimized Code** and patterns used in top product & service-based companies.

Why This Guide?

- ☐ **Save Time:** No need to browse scattered sources — this is your all-in-one prep resource.
- **Practice Smart:** Focused on **most frequently asked DSA problems**
- **Understand Concepts:** Each problem includes logic, patterns, dry-run approach, and edge cases.
- **Placement Ready:** Aligned with Coding Rounds (Online + Technical Interview Levels).

“Success isn’t just about writing code — it’s about solving problems effectively.”

Arrays

1. Two Sum — LeetCode #1

Problem Statement:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Solution Logic:

Use a hash map to store elements and their indices. For each element, check if `target - element` exists.

C++ Code:

```
#include <vector>
#include <unordered_map>

std::vector<int> twoSum(std::vector<int>& nums, int target) {
    std::unordered_map<int, int> map;
    for (int i = 0; i < nums.size(); ++i) {
        int complement = target - nums[i];
        if (map.find(complement) != map.end()) {
            return {map[complement], i};
        }
        map[nums[i]] = i;
    }
    return {};
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

2. Best Time to Buy and Sell Stock — LeetCode #121

Problem Statement:

Given an array `prices`, find the maximum profit from one buy and one sell.

Solution Logic:

Track minimum price and maximum profit while iterating once.

C++ Code:

```

#include <vector>
#include <algorithm>

int maxProfit(std::vector<int>& prices) {
    int minPrice = INT_MAX;
    int maxProfit = 0;
    for (int price : prices) {
        minPrice = std::min(minPrice, price);
        maxProfit = std::max(maxProfit, price - minPrice);
    }
    return maxProfit;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

3. Majority Element — LeetCode #169

Problem Statement:

Find the majority element that appears more than $n/2$ times in the array.

Solution Logic:

Use Boyer-Moore Voting Algorithm to find candidate in $O(n)$.

C++ Code:

```

#include <vector>

int majorityElement(std::vector<int>& nums) {
    int count = 0, candidate = 0;
    for (int num : nums) {
        if (count == 0) candidate = num;
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

4. Contains Duplicate — LeetCode #217

Problem Statement:

Check if any value appears at least twice in the array.

Solution Logic:

Use a hash set to detect duplicates.

C++ Code:

```
#include <vector>
#include <unordered_set>

bool containsDuplicate(std::vector<int>& nums) {
    std::unordered_set<int> set;
    for (int num : nums) {
        if (set.count(num)) return true;
        set.insert(num);
    }
    return false;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

5. Product of Array Except Self — LeetCode #238

Problem Statement:

Return an array where each element is the product of all others except itself, without using division.

Solution Logic:

Compute prefix and suffix products in two passes.

C++ Code:

```
#include <vector>

std::vector<int> productExceptSelf(std::vector<int>& nums) {
    int n = nums.size();
    std::vector<int> result(n, 1);

    int prefix = 1;
    for (int i = 0; i < n; ++i) {
        result[i] = prefix;
        prefix *= nums[i];
    }
}
```

```

    }

    int suffix = 1;
    for (int i = n - 1; i >= 0; --i) {
        result[i] *= suffix;
        suffix *= nums[i];
    }
    return result;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$ (excluding output)

6. Maximum Subarray (Kadane's Algorithm) — LeetCode #53

Problem Statement:

Find the contiguous subarray with the largest sum.

Solution Logic:

Iterate, track current sum and max sum, reset current sum if negative.

C++ Code:

```

#include <vector>
#include <algorithm>

int maxSubArray(std::vector<int>& nums) {
    int maxSum = nums[0], currSum = 0;
    for (int num : nums) {
        currSum = std::max(num, currSum + num);
        maxSum = std::max(maxSum, currSum);
    }
    return maxSum;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

7. Maximum Product Subarray — LeetCode #152

Problem Statement:

Find the contiguous subarray within an array which has the largest product.

Solution Logic:

Track both max and min product at each step to handle negatives.

C++ Code:

```
#include <vector>
#include <algorithm>

int maxProduct(std::vector<int>& nums) {
    int maxProd = nums[0], minProd = nums[0], result = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
        if (nums[i] < 0) std::swap(maxProd, minProd);
        maxProd = std::max(nums[i], maxProd * nums[i]);
        minProd = std::min(nums[i], minProd * nums[i]);
        result = std::max(result, maxProd);
    }
    return result;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

8. Merge Intervals — LeetCode #56

Problem Statement:

Given intervals, merge all overlapping intervals.

Solution Logic:

Sort by start, merge intervals if overlapping.

C++ Code:

```
#include <vector>
#include <algorithm>

std::vector<std::vector<int>> merge(std::vector<std::vector<int>>& intervals) {
    if (intervals.empty()) return {};
    std::sort(intervals.begin(), intervals.end());

    std::vector<std::vector<int>> merged;
```

```

merged.push_back(intervals[0]);

for (const auto& interval : intervals) {
    if (interval[0] <= merged.back()[1]) {
        merged.back()[1] = std::max(merged.back()[1], interval[1]);
    } else {
        merged.push_back(interval);
    }
}
return merged;
}

```

Time Complexity:

$O(n \log n)$

Space Complexity:

$O(n)$

9. Insert Interval — LeetCode #57

Problem Statement:

Insert a new interval into a sorted list of non-overlapping intervals and merge if necessary.

Solution Logic:

Add intervals before new interval, merge overlapping, add remaining intervals.

C++ Code:

```

#include <vector>

std::vector<std::vector<int>> insert(std::vector<std::vector<int>>& intervals,
std::vector<int>& newInterval) {
    std::vector<std::vector<int>> result;
    int i = 0, n = intervals.size();

    while (i < n && intervals[i][1] < newInterval[0]) {
        result.push_back(intervals[i++]);
    }
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = std::min(newInterval[0], intervals[i][0]);
        newInterval[1] = std::max(newInterval[1], intervals[i][1]);
        ++i;
    }
    result.push_back(newInterval);
    while (i < n) {
        result.push_back(intervals[i++]);
    }
    return result;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

10. Set Matrix Zeroes — LeetCode #73

Problem Statement:

If an element is 0, set its entire row and column to 0 in-place.

Solution Logic:

Use first row and column as markers.

C++ Code:

```
#include <vector>

void setZeroes(std::vector<std::vector<int>>& matrix) {
    bool firstRowZero = false, firstColZero = false;
    int rows = matrix.size(), cols = matrix[0].size();

    for (int r = 0; r < rows; ++r)
        if (matrix[r][0] == 0) firstColZero = true;

    for (int c = 0; c < cols; ++c)
        if (matrix[0][c] == 0) firstRowZero = true;

    for (int r = 1; r < rows; ++r) {
        for (int c = 1; c < cols; ++c) {
            if (matrix[r][c] == 0) {
                matrix[r][0] = 0;
                matrix[0][c] = 0;
            }
        }
    }

    for (int r = 1; r < rows; ++r) {
        for (int c = 1; c < cols; ++c) {
            if (matrix[r][0] == 0 || matrix[0][c] == 0) {
                matrix[r][c] = 0;
            }
        }
    }

    if (firstColZero) {
        for (int r = 0; r < rows; ++r) matrix[r][0] = 0;
    }
    if (firstRowZero) {
        for (int c = 0; c < cols; ++c) matrix[0][c] = 0;
    }
}
```

```
}
```

Time Complexity:

$O(m \times n)$

Space Complexity:

$O(1)$

11. Spiral Matrix — LeetCode #54

Problem Statement:

Return all elements of the matrix in spiral order.

Solution Logic:

Traverse the matrix layer by layer — moving right, down, left, and up — adjusting boundaries after each pass.

C++ Code:

```
#include <vector>

std::vector<int> spiralOrder(std::vector<std::vector<int>>& matrix) {
    std::vector<int> result;
    if (matrix.empty()) return result;

    int top = 0, bottom = matrix.size() - 1;
    int left = 0, right = matrix[0].size() - 1;

    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; ++c)
            result.push_back(matrix[top][c]);
        ++top;

        for (int r = top; r <= bottom; ++r)
            result.push_back(matrix[r][right]);
        --right;

        if (top <= bottom) {
            for (int c = right; c >= left; --c)
                result.push_back(matrix[bottom][c]);
            --bottom;
        }

        if (left <= right) {
            for (int r = bottom; r >= top; --r)
                result.push_back(matrix[r][left]);
            ++left;
        }
    }
    return result;
}
```

```
}
```

Time Complexity:

$O(m \times n)$

Space Complexity:

$O(m \times n)$ (for output)

12. Rotate Image (90 degrees) — LeetCode #48

Problem Statement:

Rotate an $n \times n$ 2D matrix by 90 degrees clockwise in-place.

Solution Logic:

First transpose the matrix, then reverse each row.

C++ Code:

```
#include <vector>
#include <algorithm>

void rotate(std::vector<std::vector<int>>& matrix) {
    int n = matrix.size();
    // Transpose
    for (int i = 0; i < n; ++i) {
        for (int j = i; j < n; ++j) {
            std::swap(matrix[i][j], matrix[j][i]);
        }
    }
    // Reverse each row
    for (int i = 0; i < n; ++i) {
        std::reverse(matrix[i].begin(), matrix[i].end());
    }
}
```

Time Complexity:

$O(n^2)$

Space Complexity:

$O(1)$

13. Pascal's Triangle — LeetCode #118

Problem Statement:

Generate the first `numRows` of Pascal's triangle.

Solution Logic:

Each number is the sum of the two numbers directly above it.

C++ Code:

```
#include <vector>

std::vector<std::vector<int>> generate(int numRows) {
    std::vector<std::vector<int>> triangle;

    for (int i = 0; i < numRows; ++i) {
        triangle.push_back(std::vector<int>(i + 1, 1));
        for (int j = 1; j < i; ++j) {
            triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
        }
    }
    return triangle;
}
```

Time Complexity:

$O(\text{numRows}^2)$

Space Complexity:

$O(\text{numRows}^2)$

14. Subarray Sum Equals K — LeetCode #560

Problem Statement:

Count the number of continuous subarrays whose sum equals to k .

Solution Logic:

Use prefix sums and a hash map to count how many sums have appeared.

C++ Code:

```
#include <vector>
#include <unordered_map>

int subarraySum(std::vector<int>& nums, int k) {
    std::unordered_map<int, int> prefixSumCount;
    prefixSumCount[0] = 1;
    int count = 0, sum = 0;

    for (int num : nums) {
        sum += num;
```

```

        if (prefixSumCount.find(sum - k) != prefixSumCount.end()) {
            count += prefixSumCount[sum - k];
        }
        prefixSumCount[sum]++;
    }
    return count;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

15. Missing Number — LeetCode #268

Problem Statement:

Given an array containing n distinct numbers taken from $0, 1, 2, \dots, n$, find the missing number.

Solution Logic:

Use the formula for the sum of the first n numbers and subtract the sum of array elements.

C++ Code:

```

#include <vector>

int missingNumber(std::vector<int>& nums) {
    int n = nums.size();
    int total = n * (n + 1) / 2;
    int sum = 0;
    for (int num : nums) sum += num;
    return total - sum;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

Strings

LeetCode Problem #242

16. Valid Anagram

Problem Statement:

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Solution Logic:

Count the frequency of each character in both strings and compare. If frequencies match for all characters, they are anagrams.

C++ Code:

```
#include <string>
#include <vector>
using namespace std;

bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    vector<int> count(26, 0);
    for (int i = 0; i < s.size(); ++i) {
        count[s[i] - 'a']++;
        count[t[i] - 'a']--;
    }
    for (int c : count) {
        if (c != 0) return false;
    }
    return true;
}
```

Time Complexity:

O(n), n = length of string

Space Complexity:

O(1), fixed size array of 26

LeetCode Problem #49

17. Group Anagrams

Problem Statement:

Given an array of strings, group the anagrams together. Return the answer in any order.

Solution Logic:

Sort each string to use as a key in a hashmap to group all anagrams together.

C++ Code:

```
#include <vector>
#include <string>
#include <unordered_map>
```

```

#include <algorithm>
using namespace std;

vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;
    for (string& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        groups[key].push_back(s);
    }
    vector<vector<string>> result;
    for (auto& kv : groups) {
        result.push_back(kv.second);
    }
    return result;
}

```

Time Complexity:

$O(n * k \log k)$, n = number of strings, k = max length of string

Space Complexity:

$O(n * k)$

LeetCode Problem #125

18. Valid Palindrome

Problem Statement:

Given a string s , determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Solution Logic:

Use two pointers from start and end, skip non-alphanumeric chars and compare lowercased characters.

C++ Code:

```

#include <string>
using namespace std;

bool isPalindrome(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        while (left < right && !isalnum(s[left])) left++;
        while (left < right && !isalnum(s[right])) right--;
        if (tolower(s[left]) != tolower(s[right])) return false;
        left++; right--;
    }
    return true;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

LeetCode Problem #5

19. Longest Palindromic Substring

Problem Statement:

Given a string *s*, return the longest palindromic substring in *s*.

Solution Logic:

Expand around each center (could be one or two chars) and track longest palindrome.

C++ Code:

```
#include <string>
using namespace std;

string longestPalindrome(string s) {
    int start = 0, maxLen = 1;
    int n = s.size();

    auto expand = [&](int left, int right) {
        while (left >= 0 && right < n && s[left] == s[right]) {
            left--; right++;
        }
        return right - left - 1;
    };

    for (int i = 0; i < n; ++i) {
        int len1 = expand(i, i);
        int len2 = expand(i, i+1);
        int len = max(len1, len2);
        if (len > maxLen) {
            maxLen = len;
            start = i - (len - 1) / 2;
        }
    }
    return s.substr(start, maxLen);
}
```

Time Complexity:

$O(n^2)$

Space Complexity:

$O(1)$

LeetCode Problem #14

20. Longest Common Prefix

Problem Statement:

Write a function to find the longest common prefix string amongst an array of strings. If none exists, return "".

Solution Logic:

Compare characters vertically among strings until mismatch.

C++ Code:

```
#include <vector>
#include <string>
using namespace std;

string longestCommonPrefix(vector<string>& strs) {
    if (strs.empty()) return "";
    for (int i = 0; i < strs[0].size(); ++i) {
        char c = strs[0][i];
        for (int j = 1; j < strs.size(); ++j) {
            if (i == strs[j].size() || strs[j][i] != c)
                return strs[0].substr(0, i);
        }
    }
    return strs[0];
}
```

Time Complexity:

$O(n * m)$, n = number of strings, m = min length

Space Complexity:

$O(1)$

LeetCode Problem #38

21. Count and Say

Problem Statement:

The count-and-say sequence is a sequence of digit strings defined recursively. Given an integer n , return the n th term.

Solution Logic:

Iteratively build sequence by counting consecutive digits.

C++ Code:

```
#include <string>
using namespace std;

string countAndSay(int n) {
    string result = "1";
    for (int i = 2; i <= n; ++i) {
        string temp = "";
        int count = 1;
        for (int j = 1; j <= result.size(); ++j) {
            if (j < result.size() && result[j] == result[j-1]) {
                count++;
            } else {
                temp += to_string(count) + result[j-1];
                count = 1;
            }
        }
        result = temp;
    }
    return result;
}
```

Time Complexity:

$O(n * m)$, m = length of string at step n

Space Complexity:

$O(m)$

LeetCode Problem #28

22. Implement strStr()

Problem Statement:

Return the index of the first occurrence of needle in haystack, or -1 if not found.

Solution Logic:

Use straightforward substring comparison.

C++ Code:

```
#include <string>
using namespace std;

int strStr(string haystack, string needle) {
    int n = haystack.size(), m = needle.size();
    if (m == 0) return 0;
    for (int i = 0; i <= n - m; ++i) {
        if (haystack.substr(i, m) == needle)
            return i;
    }
    return -1;
}
```

Time Complexity:

$O(n * m)$

Space Complexity:

$O(1)$

LeetCode Problem #8

23. String to Integer (atoi)

Problem Statement:

Implement atoi to convert a string to an integer with handling of whitespace, sign, invalid chars, and overflow.

Solution Logic:

Parse character by character, apply sign, check overflow limits.

C++ Code:

```
#include <string>
#include <climits>
using namespace std;

int myAtoi(string s) {
    int i = 0, n = s.size(), sign = 1;
    long long result = 0;
    while (i < n && s[i] == ' ') i++;
    if (i < n && (s[i] == '+' || s[i] == '-')) {
        sign = (s[i] == '-') ? -1 : 1;
        i++;
    }
    while (i < n && isdigit(s[i])) {
        result = result * 10 + (s[i] - '0');
        if (sign * result >= INT_MAX) return INT_MAX;
        if (sign * result <= INT_MIN) return INT_MIN;
        i++;
    }
}
```

```
    }
    return sign * result;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

LeetCode Problem #20

24. Valid Parentheses

Problem Statement:

Given a string containing only parentheses '()', '{}', '[]', determine if it is valid (all open brackets closed in correct order).

Solution Logic:

Use a stack to push opening brackets and pop when matching closing brackets occur.

C++ Code:

```
#include <string>
#include <stack>
using namespace std;

bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '{' || c == '[') {
            st.push(c);
        } else {
            if (st.empty()) return false;
            if ((c == ')' && st.top() != '(') ||
                (c == '}' && st.top() != '{') ||
                (c == ']' && st.top() != '['))
                return false;
            st.pop();
        }
    }
    return st.empty();
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

LeetCode Problem #3

25. Longest Substring Without Repeating Characters

Problem Statement:

Given a string s , find the length of the longest substring without repeating characters.

Solution Logic:

Use sliding window with a hashmap or array to track last seen indices of characters.

C++ Code:

```
#include <string>
#include <unordered_map>
using namespace std;

int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> lastSeen;
    int start = 0, maxLen = 0;
    for (int i = 0; i < s.size(); ++i) {
        if (lastSeen.count(s[i]) && lastSeen[s[i]] >= start) {
            start = lastSeen[s[i]] + 1;
        }
        lastSeen[s[i]] = i;
        maxLen = max(maxLen, i - start + 1);
    }
    return maxLen;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(\min(m, n))$ where m is charset size

Linked List

26. LeetCode #206

Reverse Linked List

Problem Statement:

Given the head of a singly linked list, reverse the list and return the reversed list.

Solution Logic:

Iterate through the list, reversing the pointers as you go. Use three pointers: `prev`, `curr`, and `next` to reverse the links.

C++ Code:

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

Time Complexity:

$O(n)$, where n is the number of nodes.

Space Complexity:

$O(1)$

27. LeetCode #21

Merge Two Sorted Lists

Problem Statement:

Merge two sorted linked lists and return it as a new sorted list.

Solution Logic:

Use two pointers to traverse both lists, append the smaller node to the merged list, and continue until all nodes are processed.

C++ Code:

```
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (l1 && l2) {
        if (l1->val < l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

Time Complexity:

$O(n + m)$, where n and m are lengths of the two lists.

Space Complexity:

$O(1)$

28. LeetCode #19**Remove Nth Node From End of List****Problem Statement:**

Given the head of a linked list, remove the n -th node from the end and return its head.

Solution Logic:

Use two pointers separated by n nodes. Move both until the second pointer reaches the end, then remove the target node.

C++ Code:

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode *first = &dummy, *second = &dummy;
    for (int i = 0; i <= n; ++i) {
        first = first->next;
    }
```

```

    }
    while (first) {
        first = first->next;
        second = second->next;
    }
    second->next = second->next->next;
    return dummy.next;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

29. LeetCode #141

Linked List Cycle Detection (Floyd's Algorithm)

Problem Statement:

Detect if a linked list has a cycle.

Solution Logic:

Use two pointers, slow and fast. Slow moves one step, fast moves two steps. If they meet, there's a cycle.

C++ Code:

```

bool hasCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

30. LeetCode #160

Intersection of Two Linked Lists

Problem Statement:

Find the node at which two singly linked lists intersect, or return `nullptr` if they do not.

Solution Logic:

Traverse both lists with two pointers; when a pointer reaches the end, switch to the other list head. They meet at intersection or end null.

C++ Code:

```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
    if (!headA || !headB) return nullptr;
    ListNode *a = headA, *b = headB;
    while (a != b) {
        a = a ? a->next : headB;
        b = b ? b->next : headA;
    }
    return a;
}
```

Time Complexity:

$O(m + n)$

Space Complexity:

$O(1)$

31. LeetCode #142

Linked List Cycle II (Start of Cycle)

Problem Statement:

Given a linked list, return the node where the cycle begins. If there is no cycle, return `nullptr`.

Solution Logic:

Use Floyd's cycle detection to find meeting point. Then, start one pointer at head and move both one step until they meet; that's the cycle start.

C++ Code:

```
ListNode *detectCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            ListNode *ptr = head;
            while (ptr != slow) {
                ptr = ptr->next;
            }
        }
    }
    return ptr;
}
```

```

        slow = slow->next;
    }
    return ptr;
}
}
return nullptr;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

32. LeetCode #234

Palindrome Linked List

Problem Statement:

Check if a singly linked list is a palindrome.

Solution Logic:

Find the middle, reverse second half, compare both halves, then restore list (optional).

C++ Code:

```

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}

bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    ListNode* secondHalf = reverseList(slow);
    ListNode* firstHalf = head;
    while (secondHalf) {
        if (firstHalf->val != secondHalf->val) return false;
        firstHalf = firstHalf->next;
    }
    return true;
}

```

```

        secondHalf = secondHalf->next;
    }
    return true;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

33. LeetCode #143

Reorder List

Problem Statement:

Reorder a linked list in the pattern: $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

Solution Logic:

Find the middle, reverse second half, then merge two lists alternating nodes.

C++ Code:

```

void reorderList(ListNode* head) {
    if (!head || !head->next) return;
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    ListNode* second = slow->next;
    slow->next = nullptr;
    ListNode* prev = nullptr;
    while (second) {
        ListNode* tmp = second->next;
        second->next = prev;
        prev = second;
        second = tmp;
    }
    ListNode* first = head;
    second = prev;
    while (second) {
        ListNode* tmp1 = first->next;
        ListNode* tmp2 = second->next;
        first->next = second;
        second->next = tmp1;
        first = tmp1;
        second = tmp2;
    }
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

STACKS AND QUEUES

LeetCode Problem #206

34. Reverse Linked List

Problem Statement:

Given the head of a singly linked list, reverse the list, and return the reversed list.

Solution Logic:

Iterate through the list, reversing the `next` pointers one by one, using three pointers: previous, current, and next.

C++ Code:

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

Time Complexity:

$O(n)$, where n is the number of nodes

Space Complexity:

$O(1)$

LeetCode Problem #21

35. Merge Two Sorted Lists

Problem Statement:

Merge two sorted linked lists and return it as a sorted list.

Solution Logic:

Use two pointers to iterate and merge nodes in ascending order.

C++ Code:

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (l1 && l2) {
        if (l1->val < l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

Time Complexity:

$O(n + m)$, n and m are lengths of the lists

Space Complexity:

$O(1)$

LeetCode Problem #141

36. Linked List Cycle

Problem Statement:

Given head of linked list, determine if it contains a cycle.

Solution Logic:

Use Floyd's cycle detection (slow and fast pointers). If they meet, cycle exists.

C++ Code:

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

LeetCode Problem #234

37. Palindrome Linked List

Problem Statement:

Check if a singly linked list is a palindrome.

Solution Logic:

Find middle, reverse second half, then compare both halves.

C++ Code:

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}

bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* secondHalf = reverseList(slow);
    ListNode* firstHalf = head;

    while (secondHalf) {
        if (firstHalf->val != secondHalf->val) return false;
        firstHalf = firstHalf->next;
        secondHalf = secondHalf->next;
    }
    return true;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

LeetCode Problem #83

38. Remove Duplicates from Sorted List

Problem Statement:

Given sorted linked list, delete all duplicates such that each element appears once.

Solution Logic:

Traverse and remove nodes with duplicate values.

C++ Code:

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* deleteDuplicates(ListNode* head) {
    ListNode* current = head;
    while (current && current->next) {
        if (current->val == current->next->val) {
            ListNode* temp = current->next;
            current->next = current->next->next;
            delete temp;
        } else {
            current = current->next;
        }
    }
    return head;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

LeetCode Problem #92

39. Reverse Linked List II

Problem Statement:

Reverse a linked list from position m to n in one pass.

Solution Logic:

Iterate to m, reverse sublist until n, then reconnect.

C++ Code:

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};
```

```

ListNode* reverseBetween(ListNode* head, int m, int n) {
    if (!head) return nullptr;
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;

    for (int i = 1; i < m; ++i) {
        prev = prev->next;
    }

    ListNode* curr = prev->next;
    for (int i = 0; i < n - m; ++i) {
        ListNode* temp = curr->next;
        curr->next = temp->next;
        temp->next = prev->next;
        prev->next = temp;
    }
    return dummy.next;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

LeetCode Problem #20

40. Valid Parentheses

Problem Statement:

Determine if a string of parentheses is valid.

Solution Logic:

Use a stack to match pairs.

C++ Code:

```

#include <stack>
#include <string>
using namespace std;

bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '{' || c == '[') {
            st.push(c);
        } else {

```

```

        if (st.empty()) return false;
        if ((c == '(' && st.top() != '(') ||
            (c == '{' && st.top() != '{') ||
            (c == '[' && st.top() != '['))
            return false;
        st.pop();
    }
}
return st.empty();
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

LeetCode Problem #232

41. Implement Queue using Stacks

Problem Statement:

Implement a queue using two stacks.

Solution Logic:

Push elements in one stack, pop from another. Transfer when needed.

C++ Code:

```

#include <stack>
using namespace std;

class MyQueue {
    stack<int> s1, s2;

public:
    void push(int x) {
        s1.push(x);
    }

    int pop() {
        if (s2.empty()) {
            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        int topVal = s2.top();
        s2.pop();
    }
};

```

```

        return topVal;
    }

    int peek() {
        if (s2.empty()) {
            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        return s2.top();
    }

    bool empty() {
        return s1.empty() && s2.empty();
    }
};

```

Time Complexity:

Amortized $O(1)$ per operation

Space Complexity:

$O(n)$

LeetCode Problem #155

42. Min Stack

Problem Statement:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Solution Logic:

Use two stacks: one normal stack and one to track minimum values.

C++ Code:

```

#include <stack>
#include <limits>
using namespace std;

class MinStack {
    stack<int> s;
    stack<int> minStack;

public:
    void push(int val) {
        s.push(val);
        if (minStack.empty() || val <= minStack.top()) {
            minStack.push(val);
        }
    }
};

```

```

    }
}

void pop() {
    if (s.top() == minStack.top()) {
        minStack.pop();
    }
    s.pop();
}

int top() {
    return s.top();
}

int getMin() {
    return minStack.top();
}
};

```

Time Complexity:

$O(1)$ per operation

Space Complexity:

$O(n)$

LeetCode Problem #347

43. Top K Frequent Elements

Problem Statement:

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements.

Solution Logic:

Count frequency with hashmap, use a min-heap (priority queue) to keep top `k`.

C++ Code:

```

#include <vector>
#include <unordered_map>
#include <queue>
using namespace std;

vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> freq;
    for (int num : nums) freq[num]++;

    auto comp = [&](int a, int b) { return freq[a] > freq[b]; };
    priority_queue<int, vector<int>, decltype(comp)> pq(comp);
}

```

```

    for (auto& [num, count] : freq) {
        pq.push(num);
        if (pq.size() > k) pq.pop();
    }

    vector<int> result;
    while (!pq.empty()) {
        result.push_back(pq.top());
        pq.pop();
    }
    reverse(result.begin(), result.end());
    return result;
}

```

Time Complexity:

$O(n \log k)$

Space Complexity:

$O(n)$

LeetCode Problem #347

44. Top K Frequent Words

Problem Statement:

Given a list of words and an integer k, return the k most frequent words. If multiple words have same frequency, return lexicographically smallest first.

Solution Logic:

Use hashmap to count frequency, then a custom comparator with priority queue.

C++ Code:

```

#include <vector>
#include <string>
#include <unordered_map>
#include <queue>
using namespace std;

vector<string> topKFrequentWords(vector<string>& words, int k) {
    unordered_map<string, int> freq;
    for (string& w : words) freq[w]++;

    auto comp = [&](const string& a, const string& b) {
        if (freq[a] == freq[b])
            return a > b; // lex order
        return freq[a] < freq[b]; // max heap by freq
    };
}

```

```

priority_queue<string, vector<string>, decltype(comp)> pq(comp);

for (auto& [w, count] : freq) {
    pq.push(w);
    if (pq.size() > k) pq.pop();
}

vector<string> result;
while (!pq.empty()) {
    result.push_back(pq.top());
    pq.pop();
}
reverse(result.begin(), result.end());
return result;
}

```

Time Complexity:

$O(n \log k)$

Space Complexity:

$O(n)$

LeetCode Problem #739

45. Daily Temperatures

Problem Statement:

Given an array of daily temperatures T , return an array $answer$ such that $answer[i]$ is the number of days until a warmer temperature.

Solution Logic:

Use a monotonic stack storing indices. Pop when current temp is higher.

C++ Code:

```

#include <vector>
#include <stack>
using namespace std;

vector<int> dailyTemperatures(vector<int>& T) {
    int n = T.size();
    vector<int> res(n, 0);
    stack<int> st; // stores indices

    for (int i = 0; i < n; ++i) {
        while (!st.empty() && T[i] > T[st.top()]) {
            int idx = st.top();
            st.pop();
            res[idx] = i - idx;
        }
        st.push(i);
    }
}

```

```

        }
        st.push(i);
    }
    return res;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

Trees & Binary Trees

LeetCode Problem #104

46. Maximum Depth of Binary Tree

Problem Statement:

Given a binary tree, find its maximum depth (number of nodes along the longest path from the root node down to the farthest leaf node).

Solution Logic:

Use recursion to find the depth of left and right subtrees, and return the $\max + 1$.

C++ Code:

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

int maxDepth(TreeNode* root) {
    if (!root) return 0;
    int leftDepth = maxDepth(root->left);
    int rightDepth = maxDepth(root->right);
    return max(leftDepth, rightDepth) + 1;
}

```

Time Complexity:

$O(n)$, where n is number of nodes

Space Complexity:

$O(h)$, h = height of tree (recursive stack)

LeetCode Problem #101

47. Symmetric Tree

Problem Statement:

Check whether a binary tree is symmetric around its center.

Solution Logic:

Use recursion to compare left and right subtrees mirror-wise.

C++ Code:

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

bool isMirror(TreeNode* t1, TreeNode* t2) {
    if (!t1 && !t2) return true;
    if (!t1 || !t2) return false;
    return (t1->val == t2->val) &&
        isMirror(t1->left, t2->right) &&
        isMirror(t1->right, t2->left);
}

bool isSymmetric(TreeNode* root) {
    if (!root) return true;
    return isMirror(root->left, root->right);
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(h)$

LeetCode Problem #112

48. Path Sum

Problem Statement:

Given a binary tree and a sum, determine if the tree has a root-to-leaf path such that adding up all values equals the sum.

Solution Logic:

DFS through the tree subtracting current node value from sum, check leaf nodes.

C++ Code:

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

bool hasPathSum(TreeNode* root, int sum) {
    if (!root) return false;
    if (!root->left && !root->right) return (sum == root->val);
    return hasPathSum(root->left, sum - root->val) || hasPathSum(root->right, sum - root->val);
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(h)$

LeetCode Problem #235

49. Lowest Common Ancestor of a BST

Problem Statement:

Given a BST and two nodes, find their lowest common ancestor.

Solution Logic:

Utilize BST properties: if both nodes smaller, go left; if both larger, go right; else current is LCA.

C++ Code:

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root) return nullptr;
    if (p->val < root->val && q->val < root->val) {
        return lowestCommonAncestor(root->left, p, q);
    }
    if (p->val > root->val && q->val > root->val) {
        return lowestCommonAncestor(root->right, p, q);
    }
    return root;
}
```

```

        return lowestCommonAncestor(root->right, p, q);
    }
    return root;
}

```

Time Complexity:

$O(h)$

Space Complexity:

$O(h)$

LeetCode Problem #104

50. Convert Sorted Array to Binary Search Tree

Problem Statement:

Given a sorted array, convert it to a height balanced BST.

Solution Logic:

Choose middle element as root recursively for balanced tree.

C++ Code:

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

TreeNode* sortedArrayToBST(vector<int>& nums, int left, int right) {
    if (left > right) return nullptr;
    int mid = left + (right - left) / 2;
    TreeNode* root = new TreeNode(nums[mid]);
    root->left = sortedArrayToBST(nums, left, mid - 1);
    root->right = sortedArrayToBST(nums, mid + 1, right);
    return root;
}

TreeNode* sortedArrayToBST(vector<int>& nums) {
    return sortedArrayToBST(nums, 0, nums.size() - 1);
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(\log n)$

LeetCode Problem #98

51. Validate Binary Search Tree

Problem Statement:

Given a binary tree, determine if it is a valid BST.

Solution Logic:

Use recursion with valid value ranges.

C++ Code:

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

bool isValidBST(TreeNode* root, long long minVal, long long maxVal) {
    if (!root) return true;
    if (root->val <= minVal || root->val >= maxVal) return false;
    return isValidBST(root->left, minVal, root->val) && isValidBST(root->right,
root->val, maxVal);
}

bool isValidBST(TreeNode* root) {
    return isValidBST(root, LONG_LONG_MIN, LONG_LONG_MAX);
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(h)$

LeetCode Problem #199

52. Binary Tree Right Side View

Problem Statement:

Return the values of the nodes you can see ordered from top to bottom when looking from the right side of a binary tree.

Solution Logic:

BFS level-order traversal, take the last node at each level.

C++ Code:

```
#include <vector>
#include <queue>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

vector<int> rightSideView(TreeNode* root) {
    vector<int> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();
            if (i == size - 1) res.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
    }
    return res;
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

LeetCode Problem #437

53. Path Sum III

Problem Statement:

Find the number of paths that sum to a given value in a binary tree. Paths do not need to start at root or end at leaf but must go downwards.

Solution Logic:

DFS with prefix sums stored in hashmap to count paths efficiently.

C++ Code:

```
#include <unordered_map>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

int dfs(TreeNode* root, int target, int currSum, unordered_map<int, int>& prefixSums) {
    if (!root) return 0;
    currSum += root->val;
    int res = prefixSums[currSum - target];
    prefixSums[currSum]++;
    res += dfs(root->left, target, currSum, prefixSums);
    res += dfs(root->right, target, currSum, prefixSums);
    prefixSums[currSum]--;
    return res;
}

int pathSum(TreeNode* root, int sum) {
    unordered_map<int, int> prefixSums;
    prefixSums[0] = 1;
    return dfs(root, sum, 0, prefixSums);
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

LeetCode Problem #98

54. Validate Binary Search Tree (Inorder Traversal)

Problem Statement:

Same as #51, validate BST using inorder traversal (iterative or recursive).

Solution Logic:

BST inorder traversal yields sorted values, check if strictly increasing.

C++ Code:

```
#include <climits>
```

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

bool inorder(TreeNode* root, long long &prev) {
    if (!root) return true;
    if (!inorder(root->left, prev)) return false;
    if (root->val <= prev) return false;
    prev = root->val;
    return inorder(root->right, prev);
}

bool isValidBST(TreeNode* root) {
    long long prev = LONG_LONG_MIN;
    return inorder(root, prev);
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(h)$

LeetCode Problem #102

55. Binary Tree Level Order Traversal

Problem Statement:

Return the level order traversal of a binary tree (from left to right, level by level).

Solution Logic:

Use BFS with a queue to iterate level by level.

C++ Code:

```

#include <vector>
#include <queue>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;

```

```

queue<TreeNode*> q;
q.push(root);
while (!q.empty()) {
    int size = q.size();
    vector<int> level;
    for (int i = 0; i < size; i++) {
        TreeNode* node = q.front(); q.pop();
        level.push_back(node->val);
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    res.push_back(level);
}
return res;
}

```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

Backtracking

56. LeetCode #23

Merge K Sorted Lists

Problem Statement:

Merge k sorted linked lists and return it as one sorted list.

Solution Logic:

Use a min-heap (priority queue) to efficiently get the smallest node among all lists.

C++ Code:

```

#include <queue>

struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

struct Compare {
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val;
    }
};

```

```

ListNode* mergeKLists(std::vector<ListNode*>& lists) {
    std::priority_queue<ListNode*, std::vector<ListNode*>, Compare> pq;
    for (auto list : lists) {
        if (list) pq.push(list);
    }

    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (!pq.empty()) {
        ListNode* node = pq.top(); pq.pop();
        tail->next = node;
        tail = tail->next;
        if (node->next) pq.push(node->next);
    }
    return dummy.next;
}

```

Time Complexity:

$O(N \log k)$, where N is total nodes, k is number of lists.

Space Complexity:

$O(k)$ for the heap.

57. LeetCode #621

Task Scheduler

Problem Statement:

Given a list of tasks and cooldown n , find the least intervals needed to finish all tasks without running the same task within cooldown.

Solution Logic:

Calculate idle slots using the frequency of the most frequent task.

C++ Code:

```

#include <vector>
#include <algorithm>

int leastInterval(std::vector<char>& tasks, int n) {
    std::vector<int> freq(26, 0);
    for (char t : tasks) freq[t - 'A']++;
    std::sort(freq.begin(), freq.end());
    int max_freq = freq[25];
    int idle_time = (max_freq - 1) * n;
    for (int i = 24; i >= 0 && idle_time > 0; i--) {
        idle_time -= std::min(freq[i], max_freq - 1);
    }
    return idle_time + tasks.size();
}

```

```

    }
    idle_time = std::max(0, idle_time);
    return idle_time + tasks.size();
}

```

Time Complexity:

$O(N + 26 \log 26) \rightarrow O(N)$

Space Complexity:

$O(1)$

58. LeetCode #46

Permutations

Problem Statement:

Return all possible permutations of a list of numbers.

Solution Logic:

Use backtracking to generate all permutations.

C++ Code:

```

#include <vector>

void backtrack(std::vector<int>& nums, std::vector<std::vector<int>>& res, int start) {
    if (start == nums.size()) {
        res.push_back(nums);
        return;
    }
    for (int i = start; i < nums.size(); ++i) {
        std::swap(nums[start], nums[i]);
        backtrack(nums, res, start + 1);
        std::swap(nums[start], nums[i]);
    }
}

std::vector<std::vector<int>> permute(std::vector<int>& nums) {
    std::vector<std::vector<int>> res;
    backtrack(nums, res, 0);
    return res;
}

```

Time Complexity:

$O(n!)$

Space Complexity:

$O(n)$ for recursion stack.

59. LeetCode #39**Combination Sum****Problem Statement:**

Find all unique combinations of candidates where the chosen numbers sum to target.

Solution Logic:

Use backtracking, allowing repeated candidates.

C++ Code:

```
#include <vector>

void backtrack(std::vector<int>& candidates, int target, int start,
              std::vector<int>& current, std::vector<std::vector<int>>& res) {
    if (target == 0) {
        res.push_back(current);
        return;
    }
    for (int i = start; i < candidates.size(); ++i) {
        if (candidates[i] > target) break;
        current.push_back(candidates[i]);
        backtrack(candidates, target - candidates[i], i, current, res);
        current.pop_back();
    }
}

std::vector<std::vector<int>> combinationSum(std::vector<int>& candidates, int
target) {
    std::sort(candidates.begin(), candidates.end());
    std::vector<std::vector<int>> res;
    std::vector<int> current;
    backtrack(candidates, target, 0, current, res);
    return res;
}
```

Time Complexity:

$O(N^{(T/M+1)})$, where T is target, M is min candidate.

Space Complexity:

$O(T/M)$

60. LeetCode #79

Word Search

Problem Statement:

Check if a given word exists in a 2D board by sequentially adjacent cells.

Solution Logic:

Use DFS backtracking marking visited cells.

C++ Code:

```
#include <vector>
#include <string>

bool dfs(std::vector<std::vector<char>>& board, std::string& word, int i, int j,
int index) {
    if (index == word.size()) return true;
    if (i < 0 || i >= board.size() || j < 0 || j >= board[0].size() || board[i][j]
!= word[index])
        return false;
    char temp = board[i][j];
    board[i][j] = '#'; // mark visited
    bool found = dfs(board, word, i+1, j, index+1) ||
                dfs(board, word, i-1, j, index+1) ||
                dfs(board, word, i, j+1, index+1) ||
                dfs(board, word, i, j-1, index+1);
    board[i][j] = temp; // revert
    return found;
}

bool exist(std::vector<std::vector<char>>& board, std::string word) {
    for (int i = 0; i < board.size(); i++) {
        for (int j = 0; j < board[0].size(); j++) {
            if (dfs(board, word, i, j, 0)) return true;
        }
    }
    return false;
}
```

Time Complexity:

$O(M \times N \times 4^L)$, where $M \times N$ is board size, L is word length.

Space Complexity:

$O(L)$ recursion stack.

61. LeetCode #131

Palindrome Partitioning

Problem Statement:

Partition a string such that every substring is a palindrome. Return all such possible partitions.

Solution Logic:

Backtracking with palindrome check.

C++ Code:

```
#include <vector>
#include <string>

bool isPalindrome(const std::string& s, int left, int right) {
    while (left < right) {
        if (s[left++] != s[right--]) return false;
    }
    return true;
}

void backtrack(std::string& s, int start, std::vector<std::string>& current,
std::vector<std::vector<std::string>>& res) {
    if (start == s.size()) {
        res.push_back(current);
        return;
    }
    for (int end = start; end < s.size(); ++end) {
        if (isPalindrome(s, start, end)) {
            current.push_back(s.substr(start, end - start + 1));
            backtrack(s, end + 1, current, res);
            current.pop_back();
        }
    }
}

std::vector<std::vector<std::string>> partition(std::string s) {
    std::vector<std::vector<std::string>> res;
    std::vector<std::string> current;
    backtrack(s, 0, current, res);
    return res;
}
```

Time Complexity:

$O(N \times 2^N)$

Space Complexity:

$O(N)$

62. LeetCode #212**Word Search II**

Problem Statement:

Given a 2D board and a list of words, find all words in the board.

Solution Logic:

Use Trie and DFS for efficient search.

C++ Code:

```
#include <vector>
#include <string>
#include <unordered_set>

struct TrieNode {
    TrieNode* children[26] = {};
    std::string word = "";
};

void insertWord(TrieNode* root, const std::string& word) {
    TrieNode* node = root;
    for (char c : word) {
        int idx = c - 'a';
        if (!node->children[idx]) node->children[idx] = new TrieNode();
        node = node->children[idx];
    }
    node->word = word;
}

void dfs(std::vector<std::vector<char>>& board, int i, int j, TrieNode* node,
std::unordered_set<std::string>& result) {
    if (i < 0 || i >= board.size() || j < 0 || j >= board[0].size() || board[i][j]
== '#' ||
!node->children[board[i][j] - 'a'])
return;
node = node->children[board[i][j] - 'a'];
if (!node->word.empty()) {
result.insert(node->word);
}
char temp = board[i][j];
board[i][j] = '#';
dfs(board, i+1, j, node, result);
dfs(board, i-1, j, node, result);
dfs(board, i, j+1, node, result);
dfs(board, i, j-1, node, result);
board[i][j] = temp;
}

std::vector<std::string> findWords(std::vector<std::vector>& board, std::vector<std::string>& words) {
TrieNode* root = new TrieNode();
for (auto& word : words) insertWord(root, word);
std::unordered_set<std::string> result_set;
```

```

for (int i = 0; i < board.size(); ++i) {
for (int j = 0; j < board[0].size(); ++j) {
dfs(board, i, j, root, result_set);
}
}
return std::vector<std::string>(result_set.begin(), result_set.end());
}

```

****Time Complexity:****

$O(M \times N \times 4^L)$

****Space Complexity:****

$O(W \times L)$, where W is number of words, L max word length.

63. LeetCode #509

Fibonacci Number (Memoization)

****Problem Statement:****

Compute the nth Fibonacci number using memoization.

****Solution Logic:****

Use recursion with caching.

****C++ Code:****

```

```cpp
#include <vector>

int fib(int n, std::vector<int>& memo) {
 if (n <= 1) return n;
 if (memo[n] != -1) return memo[n];
 memo[n] = fib(n-1, memo) + fib(n-2, memo);
 return memo[n];
}

int fib(int n) {
 std::vector<int> memo(n+1, -1);
 return fib(n, memo);
}

```

**Time Complexity:**

$O(n)$

**Space Complexity:**

$O(n)$

---

# **Dynamic Programming**

## **64. LeetCode #70**

### **Climbing Stairs**

#### **Problem Statement:**

Number of ways to climb n stairs taking 1 or 2 steps.

#### **Solution Logic:**

Dynamic programming with base cases.

#### **C++ Code:**

```
int climbStairs(int n) {
 if (n <= 2) return n;
 int a = 1, b = 2;
 for (int i = 3; i <= n; ++i) {
 int temp = a + b;
 a = b;
 b = temp;
 }
 return b;
}
```

#### **Time Complexity:**

$O(n)$

#### **Space Complexity:**

$O(1)$

---

## **65. LeetCode #64**

### **Minimum Path Sum**

#### **Problem Statement:**

Find path from top-left to bottom-right with minimum sum in a grid.

#### **Solution Logic:**

Dynamic programming summing minimum of top and left cells.

#### **C++ Code:**

```
#include <vector>
#include <algorithm>
```

```

int minPathSum(std::vector<std::vector<int>>& grid) {
 int m = grid.size(), n = grid[0].size();
 for (int i = 1; i < m; ++i) grid[i][0] += grid[i-1][0];
 for (int j = 1; j < n; ++j) grid[0][j] += grid[0][j-1];
 for (int i = 1; i < m; ++i) {
 for (int j = 1; j < n; ++j) {
 grid[i][j] += std::min(grid[i-1][j], grid[i][j-1]);
 }
 }
 return grid[m-1][n-1];
}

```

**Time Complexity:**

$O(m*n)$

**Space Complexity:**

$O(1)$

---

## 66. LeetCode #300

### Longest Increasing Subsequence

**Problem Statement:**

Find length of longest strictly increasing subsequence.

**Solution Logic:**

Use DP or binary search for optimization.

**C++ Code (DP):**

```

#include <vector>
#include <algorithm>

int lengthOfLIS(std::vector<int>& nums) {
 if (nums.empty()) return 0;
 std::vector<int> dp(nums.size(), 1);
 int max_len = 1;
 for (int i = 1; i < nums.size(); ++i) {
 for (int j = 0; j < i; ++j) {
 if (nums[i] > nums[j]) dp[i] = std::max(dp[i], dp[j] + 1);
 }
 max_len = std::max(max_len, dp[i]);
 }
 return max_len;
}

```

**Time Complexity:**

$O(n^2)$

**Space Complexity:**

$O(n)$

---

**67. LeetCode #322****Coin Change****Problem Statement:**

Find minimum number of coins to make up amount.

**Solution Logic:**

DP with bottom-up approach.

**C++ Code:**

```
#include <vector>
#include <algorithm>
#include <climits>

int coinChange(std::vector<int>& coins, int amount) {
 std::vector<int> dp(amount + 1, amount + 1);
 dp[0] = 0;
 for (int i = 1; i <= amount; ++i) {
 for (int coin : coins) {
 if (coin <= i) {
 dp[i] = std::min(dp[i], dp[i - coin] + 1);
 }
 }
 }
 return dp[amount] > amount ? -1 : dp[amount];
}
```

**Time Complexity:**

$O(\text{amount} \times \text{number\_of\_coins})$

**Space Complexity:**

$O(\text{amount})$

---

**68. LeetCode #516****Longest Palindromic Subsequence****Problem Statement:**

Find length of the longest palindromic subsequence.

**Solution Logic:**

DP on substrings comparing characters.

**C++ Code:**

```
#include <string>
#include <vector>
#include <algorithm>

int longestPalindromeSubseq(std::string s) {
 int n = s.size();
 std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));
 for (int i = n - 1; i >= 0; --i) {
 dp[i][i] = 1;
 for (int j = i + 1; j < n; ++j) {
 if (s[i] == s[j]) dp[i][j] = dp[i+1][j-1] + 2;
 else dp[i][j] = std::max(dp[i+1][j], dp[i][j-1]);
 }
 }
 return dp[0][n-1];
}
```

**Time Complexity:**

$O(n^2)$

**Space Complexity:**

$O(n^2)$

---

**69. LeetCode #72****Edit Distance (Levenshtein)****Problem Statement:**

Compute minimum operations to convert word1 to word2 (insert, delete, replace).

**Solution Logic:**

DP matrix representing operations between substrings.

**C++ Code:**

```
#include <string>
#include <vector>
#include <algorithm>

int minDistance(std::string word1, std::string word2) {
 int m = word1.size(), n = word2.size();
 std::vector<std::vector<int>> dp(m+1, std::vector<int>(n+1, 0));
 for (int i = 0; i <= m; ++i) dp[i][0] = i;
 for (int j = 0; j <= n; ++j) dp[0][j] = j;
```

```

 for (int i = 1; i <= m; ++i) {
 for (int j = 1; j <= n; ++j) {
 if (word1[i-1] == word2[j-1]) dp[i][j] = dp[i-1][j-1];
 else dp[i][j] = 1 + std::min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
 }
 }
 return dp[m][n];
}

```

### Time Complexity:

$O(m \times n)$

### Space Complexity:

$O(m \times n)$

---

## 70. LeetCode #91

### Decode Ways

#### Problem Statement:

Count ways to decode a digit string, mapping '1'-'26' to letters.

#### Solution Logic:

DP checking one and two digit decodings.

#### C++ Code:

```

#include <string>
#include <vector>

int numDecodings(std::string s) {
 if (s.empty() || s[0] == '0') return 0;
 int n = s.size();
 std::vector<int> dp(n + 1, 0);
 dp[0] = 1; dp[1] = 1;
 for (int i = 2; i <= n; ++i) {
 int one = std::stoi(s.substr(i-1,1));
 int two = std::stoi(s.substr(i-2,2));
 if (one >= 1 && one <= 9) dp[i] += dp[i-1];
 if (two >= 10 && two <= 26) dp[i] += dp[i-2];
 }
 return dp[n];
}

```

### Time Complexity:

$O(n)$

## Space Complexity:

$O(n)$

---

## 71. LeetCode #329

### Longest Increasing Path in Matrix

#### Problem Statement:

Find the longest increasing path in a matrix moving up/down/left/right.

#### Solution Logic:

DFS with memoization.

#### C++ Code:

```
#include <vector>
#include <algorithm>

int dfs(std::vector<std::vector<int>>& matrix, int i, int j,
std::vector<std::vector<int>>& memo) {
 if (memo[i][j] != 0) return memo[i][j];
 int m = matrix.size(), n = matrix[0].size();
 int dirs[4][2] = {{1,0}, {-1,0}, {0,1}, {0,-1}};
 int max_len = 1;
 for (auto& d : dirs) {
 int x = i + d[0], y = j + d[1];
 if (x >= 0 && x < m && y >= 0 && y < n && matrix[x][y] > matrix[i][j]) {
 int length = 1 + dfs(matrix, x, y, memo);
 max_len = std::max(max_len, length);
 }
 }
 memo[i][j] = max_len;
 return max_len;
}

int longestIncreasingPath(std::vector<std::vector<int>>& matrix) {
 if (matrix.empty()) return 0;
 int m = matrix.size(), n = matrix[0].size(), res = 0;
 std::vector<std::vector<int>> memo(m, std::vector<int>(n, 0));
 for (int i = 0; i < m; ++i) {
 for (int j = 0; j < n; ++j) {
 res = std::max(res, dfs(matrix, i, j, memo));
 }
 }
 return res;
}
```

**Time Complexity:**

$O(m \times n)$

**Space Complexity:**

$O(m \times n)$

---

## Advanced

### 72. LeetCode #139

#### Word Break

**Problem Statement:**

Given a string and dictionary, determine if the string can be segmented into dictionary words.

**Solution Logic:**

Use DP to check substring validity.

**C++ Code:**

```
#include <string>
#include <vector>
#include <unordered_set>

bool wordBreak(std::string s, std::vector<std::string>& wordDict) {
 std::unordered_set<std::string> dict(wordDict.begin(), wordDict.end());
 std::vector<bool> dp(s.size() + 1, false);
 dp[0] = true;
 for (int i = 1; i <= s.size(); ++i) {
 for (int j = 0; j < i; ++j) {
 if (dp[j] && dict.count(s.substr(j, i-j))) {
 dp[i] = true;
 break;
 }
 }
 }
 return dp[s.size()];
}
```

**Time Complexity:**

$O(n^2 * m)$ ,  $m$  max length of dictionary words.

**Space Complexity:**

$O(n)$

---

## 73. LeetCode #312

### Burst Balloons (Interval DP)

#### Problem Statement:

Max coins by bursting balloons wisely.

#### Solution Logic:

Use interval DP to find max coins between balloons.

#### C++ Code:

```
#include <vector>
#include <algorithm>

int maxCoins(std::vector<int>& nums) {
 int n = nums.size();
 std::vector<int> balloons(n + 2, 1);
 for (int i = 0; i < n; ++i) balloons[i + 1] = nums[i];
 std::vector<std::vector<int>> dp(n + 2, std::vector<int>(n + 2, 0));

 for (int length = 1; length <= n; ++length) {
 for (int left = 1; left <= n - length + 1; ++left) {
 int right = left + length - 1;
 for (int k = left; k <= right; ++k) {
 dp[left][right] = std::max(dp[left][right],
 dp[left][k-1] + balloons[left-1]*balloons[k]*balloons[right+1]
 + dp[k+1][right]);
 }
 }
 }
 return dp[1][n];
}
```

#### Time Complexity:

$O(n^3)$

#### Space Complexity:

$O(n^2)$

---

## 74. LeetCode #10

### Regular Expression Matching

#### Problem Statement:

Implement regex matching with support for '.' and '\*'.

- '.' matches any single character.
- '\*' matches zero or more of the preceding element.

### Solution Logic:

Use dynamic programming to compare substrings and patterns.

### C++ Code:

```
#include <string>
#include <vector>

bool isMatch(std::string s, std::string p) {
 int m = s.size(), n = p.size();
 std::vector<std::vector<bool>> dp(m+1, std::vector<bool>(n+1, false));
 dp[0][0] = true;

 // Handle patterns like a*, a*b*, a*b*c* at the start
 for (int j = 1; j <= n; ++j) {
 if (p[j-1] == '*') dp[0][j] = dp[0][j-2];
 }

 for (int i = 1; i <= m; ++i) {
 for (int j = 1; j <= n; ++j) {
 if (p[j-1] == '.' || p[j-1] == s[i-1]) {
 dp[i][j] = dp[i-1][j-1];
 } else if (p[j-1] == '*') {
 dp[i][j] = dp[i][j-2]; // zero occurrence
 if (p[j-2] == '.' || p[j-2] == s[i-1]) {
 dp[i][j] = dp[i][j] || dp[i-1][j]; // one or more occurrence
 }
 }
 }
 }
 return dp[m][n];
}
```

### Time Complexity:

$O(m \times n)$

### Space Complexity:

$O(m \times n)$

---

## 75. LeetCode #37

### Sudoku Solver

#### Problem Statement:

Write a program to solve a Sudoku puzzle by filling the empty cells.

**Solution Logic:**

Backtracking with checks for rows, columns, and 3x3 boxes.

**C++ Code:**

```
#include <vector>

bool isValid(std::vector<std::vector<char>>& board, int row, int col, char c) {
 for (int i = 0; i < 9; ++i) {
 if (board[row][i] == c) return false; // check row
 if (board[i][col] == c) return false; // check column
 if (board[3*(row/3) + i/3][3*(col/3) + i%3] == c) return false; // check
box
 }
 return true;
}

bool solveSudoku(std::vector<std::vector<char>>& board) {
 for (int i = 0; i < 9; ++i) {
 for (int j = 0; j < 9; ++j) {
 if (board[i][j] == '.') {
 for (char c = '1'; c <= '9'; ++c) {
 if (isValid(board, i, j, c)) {
 board[i][j] = c;
 if (solveSudoku(board)) return true;
 board[i][j] = '.'; // backtrack
 }
 }
 return false;
 }
 }
 }
 return true;
}
```

**Time Complexity:**

Worst case exponential, but pruning helps in practice.

**Space Complexity:**

O(1) additional space (modifies board in place).

Congratulations! You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills. Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: @SYNTAXERROR. Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved. This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.